

SAMUNDRA BUDHATHOKI | QA ENGINEER

Address: Naikap, Kathmandu, Nepal | **Contact:** db3002448@gmail.com | +977 9849558934

1. PROFESSIONAL SUMMARY

Dedicated QA Engineer focused on ensuring software quality through systematic manual testing and robust API validation. This portfolio showcases my ability to manage the end-to-end Software Testing Life Cycle (STLC), from test case design and execution to detailed bug reporting and API integrity checks. I specialize in identifying critical vulnerabilities, such as authentication bypasses and system crashes to ensure high-performance, secure software delivery.

2. PORTFOLIO AT A GLANCE

- **Manual Testing Mastery:** 21+ comprehensive test cases covering critical business logic including Login, Registration, and Search modules.
 - **API Validation:** 16+ API test cases using Postman to verify Authentication and Product CRUD operations with strict status code and response body validation.
 - **Defect Management:** 4 detailed bug reports identifying 2 **Critical** and 2 **High Priority** issues, including security-related vulnerabilities.
-

3. TECHNICAL ARSENAL

- **Tools:** Postman (API Testing), Chrome/Brave DevTools, JSON, Git.
 - **Methodologies:** Black-box Testing, SDLC/STLC, Agile Environment.
 - **Testing Techniques:** Boundary Value Analysis (BVA), Equivalence Partitioning, Positive/Negative Testing, and Input Sanitization.
-

4. CORE TESTING PRINCIPLES

My approach to quality is built on three pillars demonstrated in this portfolio:

1. **Precision in Documentation:** Every test case is documented with clear pre-conditions, logical steps, and measurable expected results.
2. **Security-First Mindset:** Beyond functional checks, I actively test for SQL injection and improper input handling to safeguard system integrity.
3. **Data Integrity:** My API testing ensures that backend services handle data accurately across all CRUD operations with proper error messaging and status codes.

Table of contents

Section	Module / Category	Details
SECTION 1: MANUAL TEST CASES	1.1 Login Module	10 Test Cases
	1.2 Search Functionality	5 Test Cases
	1.3 Registration Module	6 Test Cases
SECTION 2: BUG REPORTS	2.1 Critical Bugs	2 Bugs Identified
	2.2 High Priority Bugs	2 Bugs Identified
SECTION 3: API TEST CASES	3.1 Authentication APIs (Login)	7 Test Cases
	3.2 Registration APIs	3 Test Cases
	3.3 Product CRUD APIs	6 Test Cases

SECTION 1: MANUAL TEST CASES

1.1 LOGIN MODULE (9 Test Cases)

TC_LOGIN_001: Valid Login with Email and Password

Fields	Details
Test Case ID	TC_LOGIN_001
Test Scenario	Verify login functionality
Test Description	User should be able to login with valid email and password
Pre-conditions	User should be in login page User have registered email address
Test Steps	1. Open login page 2. Enter valid registered email 3. Enter the password 4. Click the login 5. Verify dashboard or home page
Test Data	Email: db@gmail.com password:Yesgo@1234
Expected Result	User should be logged in successfully and able to see the home page

TC_LOGIN_002: Invalid Password

Fields	Details
Test Case ID	TC_INVALID_PASSWORD_002
Test Scenario	Verify login with invalid password
Test Description	User should not be able to login with invalid password
Pre-conditions	• User should be in login page • User should have valid email

Test Steps	1. Open the login page 2. Enter valid email address 3. Enter invalid password 4. Click login 5. Verify the error message
Test Data	Email: db@gmail.com password: 123
Expected Result	• Error message is shown to the user saying "Please enter valid password" • User should be in login page

TC_LOGIN_003: Empty Password Field

Fields	Details
Test Case ID	TC_EMPTY_PASSWORD_003
Test Scenario	Verify login when the Password field is empty
Test Description	User should not be able to login if the password field is empty.
Pre-conditions	User should be in the login page.
Test Steps	1. Open the login page. 2. Enter a valid email address. 3. Leave the Password field blank. 4. Click the Login button. 5. Verify the error message
Test Data	Email: db@gmail.com Password: (empty)
Expected Result	An error message is shown to the user (e.g., "Password is required" or "Please enter your password"). User should remain on the login page.

TC_LOGIN_004: Empty Email Field

Fields	Details
Test Case ID	TC_EMPTY_EMAIL_004
Test Scenario	Verify login when the email field is empty
Test Description	User should not be able to login if the email field is empty.
Pre-conditions	User should be in the login page.
Test Steps	1. Open the login page. 2. Leave email field empty 3. Enter a valid password 4. Click the Login button. 5. Verify the error message
Test Data	Email: Password:Hello@123
Expected Result	An error message is shown to the user (e.g., "email is required" or "Please enter your email"). User should remain on the login page.

TC_LOGIN_005: Invalid Email Format

Fields	Details
Test Case ID	TC_EMAIL_FORMAT_005
Test Scenario	Verify login when the user inputs invalid email format
Test Description	User should not be able to login if the email is invalid.
Pre-conditions	User should be in the login page.

Test Steps	1. Open the login page. 2. Enter invalid email format 3. Enter a valid password 4. Click the Login button. 5. Verify the error message
Test Data	Email: dbgmail.com Password:Hello@123
Expected Result	An error message is shown to the user ("Please enter valid email"). User should remain on the login page.

TC_LOGIN_006: Unregistered Email

Fields	Details
Test Case ID	TC_UNREGISTERED_EMAIL_006
Test Scenario	Verify login when the email is unregistered in the database.
Test Description	User should not be able to login if the email is not registered.
Pre-conditions	User should be in the login page.
Test Steps	1. Open the login page. 2. Enter new or unregistered email 3. Enter a valid password 4. Click the Login button. 5. Verify the error message
Test Data	Email: new1@gmail.com Password:Hello@123
Expected Result	An error message is shown to the user (e.g., "email is not

	registered"). User should remain on the login page.
--	---

TC_LOGIN_007: SQL Injection Attempt

Fields	Details
Test Case ID	TC_LOGIN_007
Test Scenario	Verify login is protected against SQL injection
Test Description	System should sanitize inputs and prevent SQL injection
Pre-conditions	User should be in the login page.
Test Steps	1. Open the login page. 2. Enter SQL injection string in email 3. Enter a valid password 4. Click the Login button. 5. Verify system behavior
Test Data	Email: admin'-- Password:Hello@123
Expected Result	• Login fails with appropriate error • No SQL error exposed to use. • System remains stable

TC_LOGIN_008: Case Sensitivity Check

Fields	Details
Test Case ID	TC_LOGIN_008
Test Scenario	Verify email case sensitivity
Test Description	Check if login treats email as case-insensitive
Pre-conditions	User registered with db@gmail.com
Test Steps	1. Open the login page. 2. Enter email in different case (DB@GMAIL.COM) 3. Enter a valid password 4. Click the Login button. 5. Verify login success
Test Data	Email:DB@GMAIL.COM Password:Yesgo@1234
Expected Result	• Login should succeed (emails typically case-insensitive) • User redirected to dashboard

TC_LOGIN_09: Password Masking

Fields	Details
Test Case ID	TC_LOGIN_009
Test Scenario	Verify password field masking
Test Description	Password should be hidden while typing

Pre-conditions	User should be on login page
Test Steps	1. Open the login page. 2. Click on password field 3. Enter a valid password 4. Verify characters are masked (shown as dots/asterisks)
Test Data	Password:Yesgo@1234
Expected Result	• Password characters should be masked • Show/Hide password toggle should work (if present)

1.2 SEARCH FUNCTIONALITY (5 Test Cases)

TC_SEARCH_001: Valid Product Search

Fields	Details
Test Case ID	TC_VALID_SEARCH_001
Test Scenario	Verify search functionality with valid products name
Test Description	• System should display all available matching products when user enters valid product name in search field
Pre-conditions	• User should be logged in • User should be in products page • Search field and button should be visible
Test Steps	1. Login with valid credentials 2. Navigate to the products page 3. Locate the search bar 4. Enter search query:"mobile" 5. Click the Search button 6. Verify the results are displayed 7. Verify all listed products contains mobile keyword

Test Data	- email:db2@gmail.com - Password:123456 - Search Query: mobile
Expected Result	- System should filter and display all available products containing the keyword "mobile" - No unrelated products should appear - Search results should load without delay

TC_SEARCH_002: Invalid/Special Characters Search

Fields	Details
Test Case ID	TC_INVALID_SEARCH_002
Test Scenario	Verify search functionality with invalid products name
Test Description	System should display error message when the user enters invalid product name in search field
Pre-conditions	- User should be logged in - User should be on products page - Search field and button should be visible
Test Steps	- Login with valid credentials - Navigate to the products page - Locate the search bar - Enter invalid search query:”@#” - Click the Search button - Observe the error message
Test Data	- email:db2@gmail.com - Password:123456 - Search Query: :”@#”
Expected Result	- System should show error message like “Products not found” or “Invalid product name” - System shouldn’t crash - User remains on the products page

TC_SEARCH_003: Empty Search Query

Fields	Details
Test Case ID	TC_SEARCH_003
Test Scenario	Verify search with empty search field
Test Description	System should display error message when the user leaves empty field in search field
Pre-conditions	- User should be logged in - User should be on products page - Search field and button should be visible
Test Steps	- Login with valid credentials - Navigate to the products page - Locate the search bar - Enter empty search query:” ” - Click the Search button - Observe the error message
Test Data	- email:db2@gmail.com - Password:123456 - Search Query: :””
Expected Result	- System should show error message like “”Please enter search query” - System shouldn’t crash - User remains on the products page

TC_SEARCH_004: Partial Product Name

Fields	Details
Test Case ID	TC_SEARCH_004

Test Scenario	Verify search with partial product name
Test Description	System should display related products when the user enters partial name in search field
Pre-conditions	- User should be logged in - User should be on products page - Search field and button should be visible
Test Steps	1.Login with valid credentials 2.Navigate the products page 3.Locate the search bar 4.Enter empty search query:"Sams " 5.Click the Search button 6.Observe displayed products
Test Data	- email:db2@gmail.com - Password:123456 - Search Query: : "Sams"
Expected Result	- All products containing "Sams" are displayed - Results include "Samsung", "Samsung Galaxy", etc

TC_SEARCH_005: Case Insensitive Search

Fields	Details
Test Case ID	TC_SEARCH_005
Test Scenario	Verify search is case-insensitive
Test Description	System should display related products when the user enters capital or small name in search field
Pre-conditions	- User should be logged in - User should be on products page - Search field and button should be visible
Test Steps	1. Login with valid credentials 2. Navigate to the products page 3. Locate the search bar

	- Enter "MOBILE" (uppercase) - Click the Search button - Observe displayed products
Test Data	- email: db2@gmail.com - Password: 123456 - Search Query: : "MOBILE"
Expected Result	- Same results as searching "mobile" - Case should not affect search results

1.3 REGISTRATION MODULE (5 Test Cases)

TC_REG_001: Successful Registration

Fields	Details
Test Case ID	TC_REG_001
Test Scenario	Verify user registration with valid data
Test Description	User should be able to register with valid details
Pre-conditions	- User should be on registration page - Email should not be already registered
Test Steps	- Open registration page - Enter name - Enter email - Enter password - Confirm password: - Click Register button - Verify success message
Test Data	Name: Samundra

	• email:newuser@gmail.com • Password:StrongPass@123
Expected Result	• Registration successful message • User account created • Redirected to login or dashboard

TC_REG_002: Registration with Existing Email

Fields	Details
Test Case ID	TC_REG_002
Test Scenario	Verify user registration with existing email
Test Description	• User should not be able to register with already existing email
Pre-conditions	• User should be on registration page • Email already exists in database
Test Steps	1. Open registration page 2. Enter name 3. Enter already existed email 4. Enter password 5. Confirm password: 6. Click Register button 7. Verify error message
Test Data	• Name: Samundra • email:db@gmail.com(already existed) • Password:StrongPass@123
Expected Result	• System should show Error: "Email already exists" • Registration fails • User stays on registration page

TC_REG_003: Weak Password

Fields	Details
Test Case ID	TC_REG_003
Test Scenario	Verify registration functionality when user tries to register with weak password
Test Description	The system should reject the user from registering and send an error message when the user attempts to register with a weak password .
Pre-conditions	User should be on registration page
Test Steps	- Open registration page - Enter name - Enter email - Enter weak password (123) - Click Register button - Verify error message
Test Data	- Name: Samundra - email:db@gmail.com(already existed) - Password:123
Expected Result	- System should show Error: "Password must be at least 8 characters" - Registration fails - User stays on registration page

TC_REG_004: Password Mismatch

Fields	Details
Test Case ID	TC_REG_004
Test Scenario	Verify password confirmation validation
Test Description	Passwords must match

Pre-conditions	User should be on registration page
Test Steps	- Open registration page - Enter name - Enter email - Enter password: "Hello@123" - Confirm password: "Hiii@1234" - Click Register button - Verify error message
Test Data	- Name: Samundra - email: db@gmail.com - Password: "Hello@123" - Confirm password: "Hiii@1234"
Expected Result	- System should show Error: "Passwords do not match" - Registration fails - User stays on registration page

TC_REG_005: Missing Required Fields

Fields	Details
Test Case ID	TC_REG_005
Test Scenario	Verify registration functionality when user tries to register with empty fields
Test Description	The system should reject the user from registering and send an error message when the user attempts to register with empty necessary fields.
Pre-conditions	User should be on registration page
Test Steps	- Open registration page - Leave name field empty - Leave email field empty - Enter password - Click Register button - Verify error message

Test Data	• Name:''' • Email:''' • Password: hora@123
Expected Result	• System should show Error: "All fields are required" • Registration fails • User stays on registration page

SECTION 2: BUG REPORTS

2.1 CRITICAL BUGS

BUG_LOGIN_01: Authentication Bypass with Invalid Password

Fields	Details
Bug ID	BUG_LOGIN_01
Summary	User is able to login with invalid password
Description/Steps to produce	1. Open the login page 2. Enter valid email address 3. Enter invalid password 4. Click login
Pre-conditions	• User is on login page • User have valid email address
Test Data	• Email:db22@gmail.com • password:123
Actual Result	• User is able to login successfully • User is redirected to the homepage
Expected Result	• User should see the error message “invalid credentials” • User shouldn’t be redirected to the homepage • User stays on the login page
Severity/Priority	Critical/High
Environment	Brave/Chrome/mobile

BUG_SEARCH_BREAKS_01: System Crash on Special Character Search

Fields	Details
--------	---------

Bug ID	BUG_SEARCH_BREAKS_01
Summary	System crashes when user Searches with invalid/special characters
Description/Steps to produce	1. Login with valid credentials 2. Navigate to products page 3. Locate search bar 4. Enter invalid search query :”@#” 5. Click Search 6. Observe system behavior
Pre-conditions	• User is logged in • User is be on products page • Search field and button are visible
Test Data	• Email:db22@gmail.com • Password:1234567 • Search query:”@#”
Actual Result	• System crashes • User sees 404 error • Application becomes unusable until refresh
Expected Result	• User should see the error message like “invalid product name” or “product not found” • System must not crash • User stays on the products page
Severity/Priority	Critical/High
Environment	Brave/Chrome/mobile

2.2 HIGH PRIORITY BUGS

BUG_LOGIN_02: Missing Email Format Validation

Fields	Details
Bug ID	BUG_LOGIN_02

Summary	System accepts invalid email formats on login
Description/Steps to produce	1. Open login page 2. Enter email without @ symbol 3. Enter password 4. Click Login
Pre-conditions	• User is on login page
Test Data	• Email:db22@gmail.com • Password:1234567
Actual Result	• Form submits successfully
Expected Result	• User should see the error message “invalid email format” • User shouldn’t be redirected to the homepage • User stays on the login page
Severity/Priority	High/medium
Environment	Brave/Chrome/mobile

BUG_SEARCH_03: Search Results Not Filtered Correctly

Fields	Details
Bug ID	BUG_SEARCH_03
Summary	Search returns unrelated products
Description/Steps to produce	1. Login with valid credentials 2. Navigate to products page 3. Locate search bar 4. Search for “mobile” 5. Observe results include "laptop" products
Pre-conditions	• User is on login page
Test Data	• Email:db22@gmail.com • Password:1234567 • Search query:”mobile”

Actual Result	Unrelated products appear in results
Expected Result	User should not see the unrelated products name
Severity/Priority	High/medium
Environment	Brave/Chrome/mobile

SECTION 3: API TEST CASES

3.1 AUTHENTICATION APIs - LOGIN (7 Test Cases)

TC_API_LOGIN_001: Valid Login Credentials

Fields	Details
Test Case ID	TC_API_LOGIN_01
Test Scenario	Verify login functionality with valid credentials
Test Description	The system should authenticate the user when valid email and password are sent to the Login API endpoint.
Pre-conditions	- User account must exists - Postman should be installed - Api server should be running
Test Steps	- Open postman - Enter login url eg:api/login - Set request type to POST - Navigate to the body-> Raw->Json - Enter email and password: eg: <pre>{ "email": "db@gmail.com", "password": "Yesgo@1234" }</pre> - Click send - Verify the response body and status code
Test Data	"email": "db@gmail.com", "password": "Yesgo@1234"

Expected Result	• Status code :200 ok • Response body should contain : • "message": "Login Successful" • "token" field should be generated • "userId" or "user" object should be present • User should be authenticated successfully
------------------------	---

TC_API_LOGIN_002: Incorrect Password

Fields	Details
Test Case ID	TC_API_LOGIN_INCORRECT_PASSWORD_02
Test Scenario	Verify login functionality with incorrect password
Test Description	The system should reject the user and send an error message when the user attempts to login with valid email and incorrect password.
Pre-conditions	• Registered email must exist • Postman should be installed • Api server should be running
Test Steps	1. Open postman 2. Enter login url eg:api/login 3. Set request type to POST 4. Navigate to the body-> Raw->Json 5. Enter email and password: a. eg: { b. "email": "db@gmail.com", c. "password": "34" d. } 6. Click send 7. Verify the response body and status code
Test Data	• "email": "db@gmail.com", • "password": "34"

Expected Result	• Status code :401 Un authorized • Response body should contain : • "message": "Incorrect password" • "Success": "false" • No token should be generated • "userId" or "user" object should not be present • User should not be authenticated
------------------------	--

TC_API_LOGIN_003: Empty Email Field

Fields	Details
Test Case ID	TC_API_LOGIN_WITH_EMPTY_EMAIL_003
Test Scenario	Verify login functionality with Empty email
Test Description	The system should reject the user and send an error message when the user attempts to login with an empty email.
Pre-conditions	• Postman should be installed • Email field must be required as per API documentation • Api server should be running
Test Steps	1. Open postman 2. Enter login url eg:api/login 3. Set request type to POST 4. Navigate to the body-> Raw->Json 5. Enter email and password: a. eg: { b. "email": "", c. "password": "34ghijkl@" d. } 6. Click send 7. Verify the response body and status code
Test Data	• "email": "", • "password": "34ghijkl@"

Expected Result	• Status code :400 Bad request • Response body should contain : • "message": "Email field is empty" • "Success": "false" • No token should be generated • "userId" or "user" object should not be present • User should not be authenticated
------------------------	--

TC_API_LOGIN_004: Invalid Email Format

Fields	Details
Test Case ID	TC_API_LOGIN_WITH_INVALID_EMAIL_TYPE_004
Test Scenario	Verify login functionality when user tries to login with an invalid Empty format
Test Description	The system should reject the user and send an error message when the user attempts to login with an invalid email format.
Pre-conditions	• Postman should be installed • Email field must be required as per API documentation • Api server should be running
Test Steps	1. Open postman 2. Enter login url eg:api/login 3. Set request type to POST 4. Navigate to the body-> Raw->Json 5. Enter email and password: a. eg: { b. "email": "db@", c. "password": "34ghijkl@" d. } 6. Click send 7. Verify the response body and status code
Test Data	• "email": "db@", • "password": "34ghijkl@"

Expected Result	• Status code :422 Unprocessable Entity • Response body should contain : • "message": "Invalid email format" • "Success": "false" • No token should be generated • "userId" or "user" object should not be present • User should not be authenticated
------------------------	---

TC_API_LOGIN_005: Missing Required Fields

Fields	Details
Test Case ID	TC_API_REGISTRATION_WITH_MISSING_FIELDS_005
Test Scenario	Verify registration functionality when user tries to register with empty fields
Test Description	The system should reject the user from registering and send an error message when the user attempts to register with empty necessary fields.
Pre-conditions	• Postman should be installed • Email shouldn't be • Api server should be running
Test Steps	1. Open postman 2. Enter Register url eg:api/register 3. Set request type to POST 4. Navigate to the body-> Raw->Json 5. Enter name,email and password: Eg: { "name": "", "email": "", "password": "" } 6. Click send 7. Verify the response body and status code
Test Data	• "name": "" • "email": "", • "password": ""

Expected Result	• Status code :400 Bad Request • Response body should contain : • “success”:”false”, • "message": "All fields are required" • No token should be generated • New user should not be created

TC_API_LOGIN_006: Weak Password

Fields	Details
Test Case ID	TC_API_REGISTRATION_WITH_WEAK_PASSWORD_01
Test Scenario	Verify registration with weak password
Test Description	System should not able user to register with weak password
Pre-conditions	• Postman should be installed • Email used in the request must not already exist • Api server should be running
Test Steps	1. Open postman 2. Enter Register url eg:api/register 3. Set request type to POST 4. Navigate to the body-> Raw->Json 5. Enter name,email and password: Eg: { "name": "samundra" "email": "db@gmail.com", "password": "1234" } 6. Click send 7. Verify the response body and status code
Test Data	• “name”:”samundra” • "email": "db@gmail.com", • "password": "1234"
Expected Result	• Status code :422 Unprocessable Entity

	• Response body should contain : • "message": "Password must be more than 8 characters" • "Success": "false" • No token should be generated • New user should not be created
--	--

TC_API_LOGIN_007: Duplicate Email

Fields	Details
Test Case ID	TC_API_REGISTRATION_WITH_DUPLICATE_EMAIL_01
Test Scenario	Verify registration functionality when user tries to register with already registered email
Test Description	The system should reject the user from registering and send an error message when the user attempts to register with already registered email
Pre-conditions	• Postman should be installed • Email should be already existed in database • Api server should be running
Test Steps	1. Open postman 2. Enter Register url eg:api/register 3. Set request type to POST 4. Navigate to the body-> Raw->Json 5. Enter name,email and password: Eg: { "name": "samundra" "email": "db@gmail.com", "password": "123456789" } 6. Click send 7. Verify the response body and status code
Test Data	• "name": "samundra" • "email": "db@gmail.com", • "password": "123456789"

Expected Result	• Status code :409 conflict • Response body should contain : • "message": "Email already exists" • "success": "false" • No token should be generated • New user should not be created
------------------------	--

3.2 Crud APIs

TC_API_CREATE_PRODUCT: Test case for creating product with valid data

Fields	Details
Test Case ID	TC_API_CREATE_PRODUCT_01
Test Scenario	Verify creating product with valid data
Test Description	The system should create the product when valid data is sent in request body
Pre-conditions	• Postman should be installed • User should be authenticated • Api server should be running
Test Steps	1. Open postman 2. Enter create product url eg:api/create_product 3. Set request type to POST 4. Navigate to the body-> Raw->Json 5. Enter name,quantity,price Eg: { "name": "samsung1" "qty": "3", "price": "12000" } 6. Click send 7. Verify the response body and status code
Test Data	• "name": "samsung1" • "qty": 3,

	• "price": 12000
Expected Result	• Status code :201 created • Response body should contain : • "message": "product created successfully" • "success": "True" • New product id should be generated

TC_API_CREATE_PRODUCT: Test case for creating product with missing fields

Fields	Details
Test Case ID	TC_API_CREATE_PRODUCT_02
Test Scenario	Verify creating product with missing required fields
Test Description	The system should not create the product and sends error message when missing data is sent in request body
Pre-conditions	• Postman should be installed • User should be authenticated • Api server should be running
Test Steps	1. Open postman 2. Enter create product url eg:api/create_product 3. Set request type to POST 4. Navigate to the body-> Raw->Json 5. Enter name,quantity,price Eg: { "name": "samsung1" } 6. Click send 7. Verify the response body and status code
Test Data	• "name": "samsung1"

Expected Result	• Status code :400 bad request • Response body should contain : • "message": "All fields are required", • "success": "false" • New product id should not be generated

TC_API_GET_PRODUCT: Test case for getting products

Fields	Details
Test Case ID	TC_API_GET_PRODUCTS_03
Test Scenario	Verify getting product with correct product ID
Test Description	The system should give product details when correct product ID is sent in request body
Pre-conditions	• Postman should be installed • User should be authenticated • Api server should be running • Product with product id 1 should exist
Test Steps	1. Open postman 2. Enter Get product url eg:api/get_product/1 3. Set request type to GET 4. Click send 5. Verify the response body and status code
Test Data	• Url: api/get_product • product_id:1
Expected Result	• Status code :200 OK • Response body should contain: {

	<pre> "success": true, "product": { "name": "Samsung01", "qty": 3, "price": 12000 } } </pre>
--	--

TC_API_GET_PRODUCT: Test case for getting products with invalid id

Fields	Details
Test Case ID	TC_API_GET_PRODUCT_WITH_INVALID_ID_04
Test Scenario	Verify fetching product with Incorrect product ID
Test Description	The system should not give product details and sends error message when incorrect product ID is sent in request body
Pre-conditions	• Postman should be installed • User should be authenticated • Api server should be running • Product with id 99 should not exist
Test Steps	1. Open postman 2. Enter Get product url eg:api/get_product/999 3. Set request type to GET 4. Click send 5. Verify the response body and status code
Test Data	• Url: api/get_product • product_id:999
Expected Result	• Status code :404 Not found • Response body should contain: <pre> { "success": false, "messege": "Products does not exist" } </pre> • Product details should not be displayed

TC_API_UPDATE_PRODUCT: Test case for updating products

Fields	Details
Test Case ID	TC_API_UPDATE_PRODUCT_01
Test Scenario	Verify updating product with valid product ID
Test Description	The system should update product details when correct product ID with data is sent in request body
Pre-conditions	• Postman should be installed • User should be authenticated • Api server should be running • Product with product id 1 should exist
Test Steps	1. Open postman 2. Enter update product url eg:api/update_product/1 3. Set request type to PUT 4. Enter product details to update<pre>{ "name": "Iphone7", "qty": 4, "price": 20000 }</pre> 5. Click send 6. Verify the response body and status code
Test Data	• Url: api/update_product • product_id:1
Expected Result	• Status code :200 OK • Response body should contain:<pre>{ "success": true, "messege": "product details updated successfully" "product": { "id": 1, "name": "Iphone7", "qty": 4, "price": 20000 } }</pre>

TC_API_DELETE_PRODUCT: Test case for deleting products

Fields	Details
Test Case ID	TC_API_DELETE_PRODUCT_01
Test Scenario	Verify deleting product with valid product ID
Test Description	The system should delete product when correct product ID is sent in request body
Pre-conditions	• Postman should be installed • User should be authenticated • Api server should be running • Product with product id 1 should exist
Test Steps	1. Open postman 2. Enter delete product url eg:api/delete_product/1 3. Set request type to DELETE 4. Click send 5. Verify the response body and status code
Test Data	• Url: api/delete_product • product_id:1
Expected Result	• Status code :200 OK • Response body should contain:<pre>{ "success": true, "messege": "product deleted successfully" }</pre> • Product should not exists